

■# Relatório Técnico - Modelagem Matemática GNSS do HoloPass

Disciplina: Differentiated Problem Solving**Projeto:** HoloPass - Global Solution 2026 - Indústria Espacial**Integrante:** Thiago Souza de Lima - RM 568732

1. Problema e contexto

O HoloPass é uma pulseira inteligente para transporte público. Ela paga a passagem por NFC e usa GNSS para identificar a estação mais próxima, calcular rotas e apoiar decisões sobre áreas urbanas mal atendidas por estações.

O problema matemático estudado é: como transformar coordenadas, sinais e distâncias de satélite em informações úteis para mobilidade urbana? A resposta exige funções capazes de representar distância na Terra, posição estimada, perda de sinal e redução de erro conforme aumenta a quantidade de satélites visíveis.

2. Variáveis do modelo

- lat , lon : latitude e longitude do usuário ou estação.
- d : distância entre dois pontos na superfície terrestre, em quilômetros.
- x , y : coordenadas planas usadas na trilateração didática.
- r_1 , r_2 , r_3 : distâncias até satélites ou antenas de referência.
- f : frequência do sinal GNSS, em MHz.
- n : número de satélites visíveis no fix.
- $E(n)$: erro posicional estimado em metros.

3. Funções construídas

3.1 Função trigonométrica - Haversine

A função de Haversine calcula a distância real entre dois pontos da Terra considerando sua curvatura. Ela foi escolhida porque o app trabalha com latitude e longitude reais, e uma distância plana simples gera erro em trajetos urbanos.

Domínio: latitudes entre -90 e 90 graus; longitudes entre -180 e 180 graus.**Imagem:** distâncias reais maiores ou iguais a zero.**Comportamento:** a distância cresce conforme aumenta a separação angular entre os pontos.**Interpretação:** permite detectar a estação mais próxima por GNSS com rigor geométrico.

3.2 Função polinomial - trilateração

A trilateração usa equações quadráticas de circunferências para estimar posições a partir de distâncias até referências. No contexto espacial, ela representa a base matemática simplificada do posicionamento por satélite.

Domínio: coordenadas e raios reais positivos.

Imagem: pontos possíveis de interseção entre circunferências.

Comportamento: diferenças entre quadrados de distâncias linearizam parte do sistema.

Interpretação: mostra por que múltiplos satélites são necessários para localizar o usuário.

3.3 Função logarítmica - perda de sinal no espaço livre

A função FSPL representa a perda de potência do sinal conforme distância e frequência aumentam. Ela foi escolhida porque sinais GNSS chegam fracos à superfície e precisam de antena/receptor adequados.

Domínio: distância maior que zero e frequência maior que zero.

Imagem: perda em decibéis.

Comportamento: crescente e logarítmico; dobrar a distância aumenta a perda, mas não de forma linear.

Interpretação: reforça a importância da precisão informada pelo navegador no app.

3.4 Função exponencial - erro posicional didático

A função exponencial modela, de forma didática, a redução do erro conforme mais satélites ficam disponíveis. Ela não afirma medições reais; serve para visualizar o comportamento esperado.

Domínio: número inteiro positivo de satélites visíveis.

Imagem: erro positivo em metros.

Comportamento: decrescente; o ganho é maior no início e estabiliza depois.

Interpretação: ajuda a explicar por que um fix com mais satélites tende a ser mais confiável.

4. Implementação em Python

O arquivo `modelo_gnss.py` implementa cálculos e gráficos para as quatro funções. O programa usa funções com parâmetros e retorno, imprime análise textual e salva gráficos com título, eixos e identificação.

Quando `matplotlib` e `numpy` estão instalados, os gráficos são gerados em PNG. Se essas bibliotecas não estiverem disponíveis, o código usa fallback em SVG para manter a entrega executável.

5. Gráficos e resultados esperados

Ao executar ``python modelo_gnss.py``, o programa gera:

- ``grafico_1_haversine``: distância $\hat{A}$ esta $\hat{S}$ $\hat{A}$ o $\hat{S}$ variando latitude.
- ``grafico_2_trilateracao``: interse $\hat{S}$ $\hat{A}$ o did $\hat{A}$ tica de circunfer $\hat{A}$ ncias.
- ``grafico_3_fspl``: perda de sinal em fun $\hat{S}$ $\hat{A}$ o da distância.
- ``grafico_4_exponencial``: erro posicional conforme sat $\hat{A}$ lites vis $\hat{A}$ veis.

A saída textual apresenta domínio, imagem, crescimento/decrescimento e interpretação física de cada modelo.

6. Conclusão técnica

O modelo atende ao briefing porque conecta matemática, Python e Indústria Espacial de forma coesa. Haversine resolve a distância GNSS do app, a trilateração explica a lógica de posicionamento, FSPL justifica limitações de sinal e a função exponencial comunica o impacto da quantidade de satélites.

Essas funções sustentam a proposta do HoloPass: uma solução de mobilidade que usa tecnologia espacial para pagamento, localização, planejamento urbano e inclusão no transporte público.

7. Como executar

```
python differentiated-problem-solving/modelo_gnss.py
```

Dependências opcionais para gráficos PNG:

```
pip install matplotlib numpy
```

Sem essas dependências, o script ainda gera gráficos SVG automaticamente.